LAPORAN PRAKTIKUM

Pengantar Berorientasi Objek

Diajukan untuk memenuhi salah satu tugas praktikum
Pengantar Berorientasi Objek



Disusun Oleh:

Najwan Rachman (251511024)

Link Repository:

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2026

Praktikum I Employee, Manager, dan Sortable

1.Employee

Class Employee dibuat dengan beberapa atribut private yaitu name, salary, hireDay, hireMonth, dan hireYear.

Class ini memiliki beberapa method, yaitu:

- print()

- raiseSalary()

- hireYear()

- getSalary()

Penggunaan atribut private dilakukan agar data pada Employee tetap terjaga dan diakses melalui method yang disediakan.

Code Employee

```
public class Employee {
   private String name;
   private double salary;
   private int hireDay;
   private int hireMonth;
   private int hireYear;

   public Employee(String name, double salary,
                   int day, int month, int year) {
      this.name = name;
      this.salary = salary;
      this.hireDay = day;
      this.hireMonth = month;
      this.hireYear = year;
   }

   public void print() {
      System.out.println(name + " " + salary + " " + hireYear());
```

```
    }

    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }

    public int hireYear() {
        return hireYear;
    }

    public double getSalary() {
        return salary;
    }
}
```

2.Pengujian Employee

Tiga object Employee dibuat dengan data yang berbeda. Setiap object kemudian mendapatkan kenaikan salary sebesar 5%.

Hasil Pengujian

Antonio Rossi 2100000.0 1989

Maria Bianchi 2625000.0 1991

Isabel Vidal 3150000.0 1993

3.Manager sebagai Subclass Employee

Class Manager dibuat sebagai turunan dari Employee menggunakan:

public class Manager extends Employee

Manager memiliki atribut tambahan yaitu secretaryName. Method raiseSalary() juga dioverride sehingga Manager mendapatkan tambahan bonus berdasarkan lama bekerja. Modul Praktikum PBO Pertemuan 5…

Pada pengujian berikutnya, Maria Bianchi diubah dari Employee menjadi Manager.

Hasil Pengujian

```
PS C:\Semester 3\PBO\week 5\src> javac *.java
PS C:\Semester 3\PBO\week 5\src> java EmployeeTest
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
```

Perbedaan salary Maria terjadi karena object tersebut sudah menjadi Manager dan menggunakan raiseSalary() milik Manager.

4.Abstract Class Sortable

Sortable dibuat sebagai abstract class dengan method abstract compare().

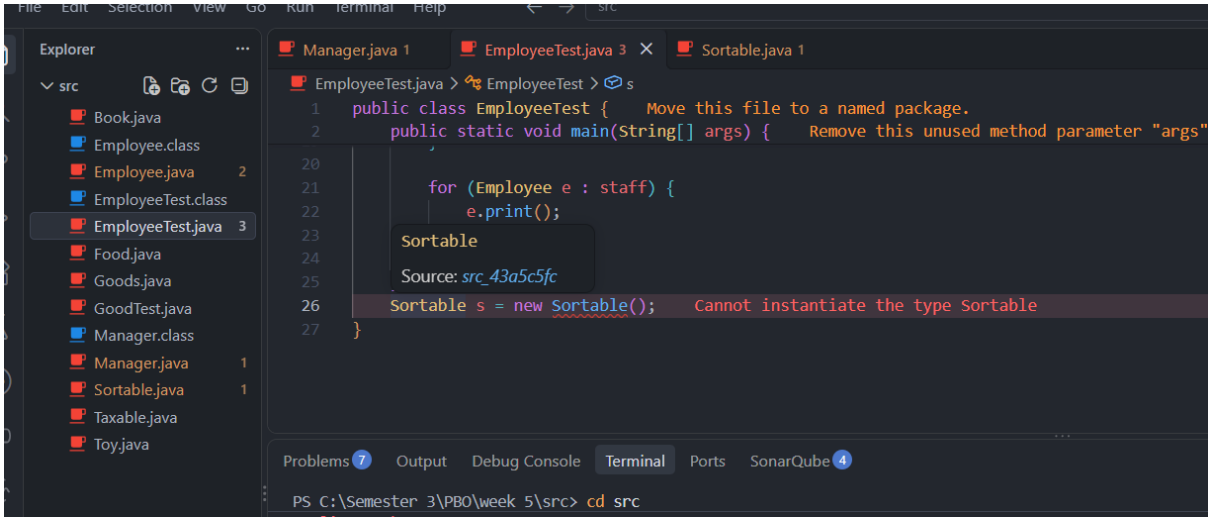
public abstract class Sortable {

public abstract int compare(Sortable other);

}

Karena Sortable merupakan abstract class, object dari class tersebut tidak dapat dibuat secara langsung.

menghasilkan error:



5.Employee extends Sortable

Selanjutnya Employee diubah menjadi public class Employee extends Sortable

Pada percobaan pertama, method compare() belum ditambahkan. Saat program di-compile, muncul error:

Employee is not abstract and does not override abstract method compare(Sortable) in Sortable

Error tersebut terjadi karena Sortable memiliki abstract method compare(), sehingga Employee harus menyediakan implementasi method tersebut.

Setelah itu ditambahkan:

```
@Override
public int compare(Sortable other) {
    Employee otherEmployee = (Employee) other;

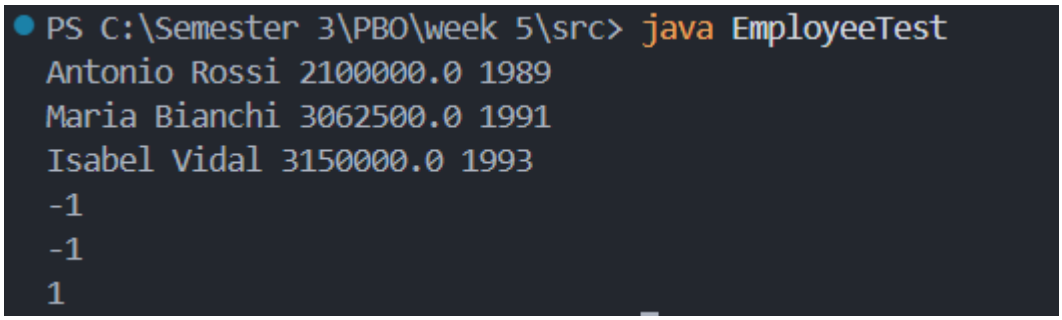
    if (getSalary() < otherEmployee.getSalary()) return -1;
    if (getSalary() > otherEmployee.getSalary()) return 1;

    return 0;
}
```

6.Pengujian compare()

Pengujian dilakukan dengan membandingkan salary beberapa object Employee.

Hasil:



Artinya:

Antonio dibandingkan dengan Maria menghasilkan -1 karena salary Antonio lebih kecil.

Maria dibandingkan dengan Isabel menghasilkan -1 karena salary Maria lebih kecil.

Isabel dibandingkan dengan Antonio menghasilkan 1 karena salary Isabel lebih besar.

7. Inheritance Chain

Hubungan class yang digunakan pada praktikum adalah:

Manager ke Employee ke Sortable

Manager tidak perlu menulis extends Sortable karena Manager sudah mewarisi Employee, sedangkan Employee merupakan subclass dari Sortable. Oleh karena itu, method compare() juga dapat digunakan oleh Manager.

8.Percobaan Multiple Extends

Dicoba deklarasi:

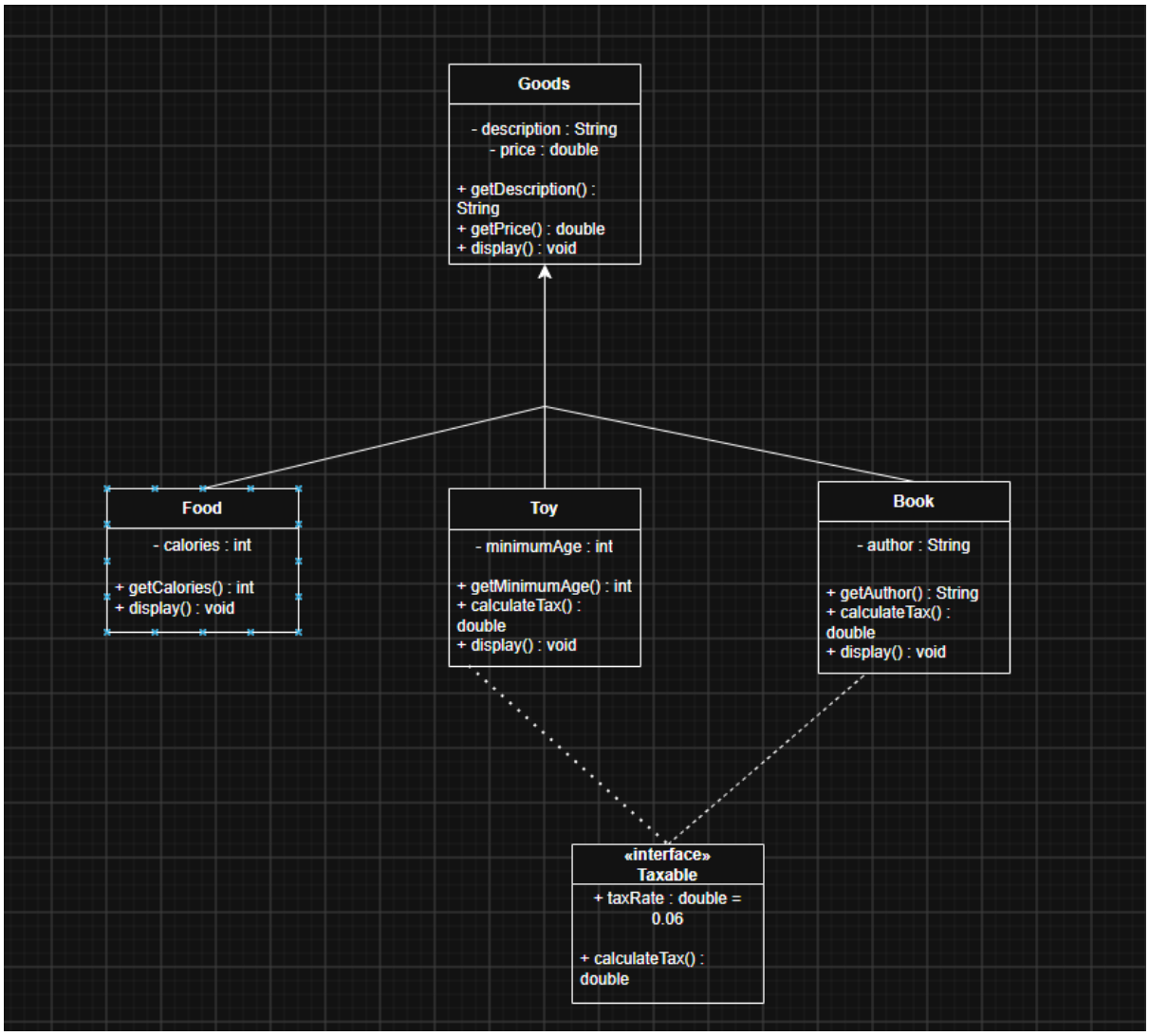
public class Manager extends Employee extends Sortable

Hasilnya error karena Java tidak mengizinkan sebuah class melakukan extends terhadap dua class.

```
PS C:\Semester 3\PBO\week 5\src> javac *.java
Manager.java:4: error: '{' expected
public class Manager extends Employee extends Sortable {
                                               ^
1 error
```

Praktikum II Goods dan Taxable

Pada bagian kedua dibuat hierarchy Goods yang memiliki tiga subclass:



Food, Toy, dan Book merupakan subclass dari Goods. Sedangkan Toy dan Book juga mengimplementasikan interface Taxabl

9.Goods

Goods memiliki atribut:

private String description;

private double price;

serta method:

getDescription()

getPrice()

display()

Atribut tersebut ditempatkan pada Goods karena ketiga jenis barang memiliki informasi umum berupa description dan price

10.Food

Food merupakan subclass dari Goods:

public class Food extends Goods

Food memiliki atribut tambahan:

private int calories;

Method display() dioverride untuk menampilkan informasi dari Goods serta jumlah calories.

11.Interface Taxable

Interface Taxable dibuat sebagai berikut:

```
public interface Taxable {
    double taxRate = 0.06;
    double calculateTax();
}
```

Percobaan membuat object langsung:

Taxable t = new Taxable();

menghasilkan error:

Taxable is abstract; cannot be instantiated

```
PS C:\Semester 3\PBO\week 5\src> javac *.java
GoodsTest.java:4: error: Taxable is abstract; cannot be instantiate
d
        Taxable t = new Taxable();
                    ^
1 error
```

Percobaan ini menunjukkan bahwa Taxable tidak dibuat sebagai object langsung, tetapi digunakan sebagai contract untuk class yang mengimplementasikannya

12.Toy

Toy menggunakan:

public class Toy extends Goods implements Taxable

Artinya Toy merupakan subclass dari Goods dan sekaligus memenuhi contract dari Taxable.

Method pajaknya:

```
@Override
public double calculateTax() {
    return getPrice() * taxRate;
}
```

Untuk harga Toy sebesar 120000, pajaknya adalah:

120000 × 0.06 = 7200

13.Book

Book juga menggunakan:

public class Book extends Goods implements Taxable

Book memiliki atribut tambahan:

private String author;

dan mengimplementasikan calculateTax().

Untuk harga Book sebesar 90000:

90000 × 0.06 = 5400

14.Pengujian GoodsTest

Object yang digunakan:

```
Food food = new Food("Bread", 15000, 250);
Toy toy = new Toy("Robot", 120000, 8);
Book book = new Book("Java Basics", 90000, "A. Programmer");
```

Hasil program:

```
PS C:\Semester 3\PBO\week 5\src> java GoodsTest
Description : Bread
Price : 15000.0
Calories : 250

Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0

Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0

Toy tax : 7200.0
Book tax : 5400.0
```

15.Pengujian Contract Taxable

Pada percobaan berikutnya, method calculateTax() dihapus dari Toy, tetapi deklarasi:

implements Taxable

tetap dipertahankan.

Compiler menghasilkan error:

Toy is not abstract and does not override abstract method

calculateTax() in Taxable

Selain itu, pemanggilan calculateTax() di dalam display() juga menghasilkan error karena method tersebut sudah tidak tersedia.

Dari percobaan tersebut dapat dilihat bahwa class yang menggunakan implements Taxable harus menyediakan method calculateTax(). Modul Praktikum PBO Pertemuan 5…

Setelah pengujian selesai, method calculateTax() dikembalikan sehingga program dapat di-compile dan dijalankan kembali.

16.Perbandingan Abstract Class dan Interface

Pertanyaan	**Sortable**	**Taxable**
Jenis	Abstract class	Interface

Pertanyaan	Sortable	Taxable
Dapat dibuat object langsung?	Tidak	Tidak
Digunakan dengan	extends	implements
Method pada praktikum	compare()	calculateTax()
Penggunaan	Menjadi dasar Employee	Contract untuk Toy dan Book
Bisa digabung dengan parent class?	Class hanya dapat extends satu class	Interface dapat diimplementasikan bersama extends

Pada praktikum, Sortable digunakan sebagai dasar untuk membuat kewajiban compare() pada Employee, sedangkan Taxable digunakan sebagai contract yang harus dipenuhi oleh Toy dan Book.

17.Refleksi

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Sortable berguna sebagai class dasar untuk menentukan bahwa class turunannya harus memiliki method compare(). Jadi walaupun object Sortable tidak dibuat langsung, class tersebut tetap digunakan oleh Employee.

2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Compiler menghasilkan error karena Employee belum memenuhi abstract method compare() yang berasal dari Sortable.

3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Karena Manager merupakan subclass dari Employee dan Employee merupakan subclass dari Sortable. Method compare() diwariskan melalui inheritance.

4. Mengapa class Manager extends Employee extends Sortable tidak diperlukan dan tidak valid di Java?

Karena Employee sudah menjadi subclass Sortable. Java juga tidak mengizinkan satu class memiliki lebih dari satu superclass menggunakan extends.

5. Apa yang dijanjikan Toy ketika menulis implements Taxable?

Toy menyatakan bahwa dirinya memenuhi contract dari Taxable dan harus menyediakan method calculateTax().

6. Mengapa Goods menjadi superclass Food, Toy, dan Book, sedangkan Taxable menjadi interface?

Goods berisi data dan behavior yang sama-sama dibutuhkan oleh Food, Toy, dan Book. Taxable hanya menentukan behavior tertentu, yaitu perhitungan pajak, sehingga lebih sesuai digunakan sebagai interface.

7. Apa bukti dari compiler bahwa interface merupakan contract?

Ketika calculateTax() dihapus dari Toy tetapi Toy tetap implements Taxable, compiler menghasilkan error karena method tersebut wajib dibuat.

8. Lengkapi kalimat

Abstract class membantu saya ketika membutuhkan class dasar yang menjadi acuan bagi subclass, sedangkan interface membantu saya ketika membutuhkan sebuah contract yang harus dipenuhi oleh class tertentu.

18.Kesimpulan

Dari praktikum ini dapat dipahami bahwa abstract class dan interface memiliki fungsi yang berbeda. Abstract class dapat digunakan sebagai dasar untuk class lain dan dapat memiliki abstract method yang wajib diimplementasikan oleh subclass. Hal ini diterapkan pada Sortable dan Employee.

Interface digunakan untuk menentukan contract yang harus dipenuhi oleh class yang menggunakannya. Pada praktikum ini, Taxable digunakan oleh Toy dan Book untuk menyediakan method calculateTax().

Selain itu, praktikum juga menunjukkan bahwa inheritance dapat membentuk rantai seperti Sortable → Employee → Manager, sehingga subclass dapat menggunakan method yang berasal dari superclass tanpa perlu menuliskan extends kembali.